

Parul® University | NAAC A++

FACULTY OF ENGINEERING & TECHNOLOGY

BACHELOR OF TECHNOLOGY

Data Structure and Algorithms

(03010503PC02)

3rd SEMESTER

COMPUTER SCIENCE & ENGINEERING

DEPARTMENT

Laboratory Manual

DATA STRUCTURE AND ALGORITHM PRACTICAL BOOK

COMPUTER SCIENCE AND ENGINEERING DEPARTMENT

CERTIFICATE

This is to certify that ***Mr. Bikash Kumar Yadav*** *with enrollment no.* *2503031050477* *has successfully completed her laboratory experiments in* ***Data Structure and Algorithm (03010503PC02)*** *from the department of computer science and engineering during the academic year 2026-27.*

Date of Submission:..... Staff In charge:.....

Head of Department:.....

INDEX

SR. NO	PROGRAM	PAGE. NO	DATE OF PERFORM ANCE	DATE OF SUBMISSION	MARKS	SIGN
1	Implement Stack and its operations like (creation push pop traverse peek search) using linear data structure					
2	Implement Infix to Postfix Expression Conversion using Stack.					
3	Implement Postfix evaluation using Stack.					
4	Implement Towers of Hanoi using Stack.					
5	Implement queue and its operations like enqueue, dequeue, traverse, search.					
6	Implement Single Linked lists and its operations (creation insertion deletion traversal search reverse)					
7	Implement Double Linked lists and its operations (creation insertion deletion traversal search reverse)					
8	Implement Circular Linked lists and its operations (creation insertion deletion traversal search reverse)					
9	Implement Linear Search, binary search and interpolation search.					
10	Implement Bubble sort, selection sort, Insertion sort, quick sort, merge sort.					
11	Implement Binary search Tree and its operations (creation, insertion, deletion).					
12	Implement Traversals Preorder, Inorder, Postorder on BST.					
13	Implement Graphs and represent using adjacency list and adjacency matrix and implement basic operations with traversals (BFS and DFS).					

Practical 1

Aim: Implement Stack and its operations like (creation push pop traverse peek search) using linear data structure

1. Construct a stack of 100-capacity
2. Push elements
3. Pop elements
4. Top element
5. Traverse element
6. Search element

Theory:

1. Introduction to Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The element inserted last is the first one to be removed. All insertion and deletion operations are performed at a single end called the **Top**.

2. Array Representation of Stack

When implemented using a linear array:

- A fixed-size array stores the elements.
- A variable top acts as a pointer to the index of the topmost element.
- **Initial State:** When the stack is empty, top is initialized to -1.

3. Core Stack Operations

- **Creation:** Allocates memory for the stack and initializes top = -1.
- **Push:** Inserts an element at the top.
  - *Condition:* Must check for **Stack Overflow** (top == MAX - 1) before inserting.
- **Pop:** Removes and returns the top element.
  - *Condition:* Must check for **Stack Underflow** (top == -1) before deleting.
- **Peek:** Returns the topmost element without removing it. (Requires an underflow check).
- **Traverse:** Visually displays the stack elements by iterating from top down to 0.
- **Search:** Compares a target element sequentially from top to 0 to find its position.

4. Summary Table & Complexity

Operation	Condition / Check
Push (Insertion)	**Overflow if** `top == MAX - 1`
Pop (Deletion)	**Underflow if** `top == -1`
Peek (View Top)	**Underflow if** `top == -1`
Traverse / Search	**N/A**

5. Applications

- Function call management (Recursion stack).
- Undo/Redo operations in text editors.
- Expression parsing and conversion (e.g., Infix to Postfix).

Code Implementation:

```
#include <iostream>

using namespace std;

#define MAX 100

class Stack {
    int arr[MAX];
    int top;
    public:
        Stack() {
            top = -1;
        }
```

```
    bool isFull() {
        return top == MAX
- 1;
    }
    bool isEmpty() {
        return top == -1;
    }

    void push(int val) {
        if (isFull()) {
            cout << "Stack
Overflow!\n";
            return;
        }
        arr[++top] = val;
        cout << val << "
pushed onto stack.\n";
    }

    void pop() {
        if (isEmpty()) {
            cout << "Stack
Underflow!\n";
            return;
        }
        cout << arr[top--]
<< " popped from
stack.\n";
    }

    void peek() {
        if (isEmpty()) {
            cout << "Stack is
empty.\n";
            return;
        }
        cout << "Top
element: " << arr[top]
<< "\n";
    }

    void traverse() {
        if (isEmpty()) {
            cout << "Stack is
empty.\n";
            return;
        }
        cout << "Stack (top
to bottom): ";
        for (int i = top; i >=
0; i--)
```

```
            cout << arr[i] <<
" ";
        cout << "\n";
    }

    void search(int val) {
        for (int i = top; i >=
0; i--)
            if (arr[i] == val)
{
                cout << val <<
" found at position " <<
(top - i + 1) << " from
top.\n";
                return;
            }
        cout << val << "
not found in stack.\n";
    }
};

int main() {
    Stack s;
    int choice, val;
    do {
        cout << "\n--- Stack
Menu ---\n";
        cout << "1.Push
2.Pop 3.Peek 4.Traverse
5.Search 0.Exit\n";
        cout << "Enter
choice: ";
        cin >> choice;
        switch (choice) {
            case 1:
                cout << "Enter
value: ";
                cin >> val;
                s.push(val);
                break;
            case 2:
                s.pop();
                break;
            case 3:
                s.peek();
                break;
            case 4:
                s.traverse();
                break;
            case 5:
                cout << "Enter
```

```
value to search: ";
                cin >> val;
                s.search(val);
                break;
        }
    } while (choice != 0);
    return 0;
}
```

Sample Input

```
1
2
3
4
5
3
2
0
```

Your Output

```
--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice: Enter value: 2 pushed onto stack.

--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice: Top element: 2
```

Your Output

```
--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice: Stack (top to bottom): 2

--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice: Enter value to search: 3 not found in stack.
```

```
--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice: 2 popped from stack.

--- Stack Menu ---
1.Push 2.Pop 3.Peek 4.Traverse 5.Search 0.Exit
Enter choice:
```

Practical 2

Aim: Implement Infix to Postfix Expression Conversion using Stack.

Theory:

1. Types of Arithmetic Expressions

In computer science, arithmetic expressions can be written in three notation formats depending on the position of the operators relative to their operands**:**

- **Infix Notation:** The operator is placed *between* the operands. (e.g., A + B). This is how humans naturally write expressions, but it requires parentheses to define precedence.
- **Postfix Notation (Reverse Polish Notation):** The operator is placed *after* the operands. (e.g., AB+).
- **Prefix Notation (Polish Notation):** The operator is placed *before* the operands. (e.g., +AB).

While infix notation is easy for humans to read, it is complex for computers to evaluate because of parentheses and operator precedence (such as multiplication having higher priority than addition).

Postfix notation is ideal for computers because:

- It does not require parentheses.
- Operators appear in the exact order of execution.
- It can be evaluated in a single scan from left to right using a stack.

3. Role of Stack in Conversion

A stack is used to temporarily hold operators (and open parentheses) during the conversion process. Operands (like letters or numbers) bypass the stack and are directly appended to the output expression.

Operators are pushed onto or popped from the stack based on their precedence and associativity:

Operator	Precedence
^ (Exponentiation)	High (3)
*, / (Multiplication, Division)	Medium (2)
+, - (Addition, Subtraction)	Low (1)

4. Algorithm for Infix to Postfix Conversion

Scan the infix expression from left to right:

1. If the character is an operand: Append it directly to the postfix output.
2. If the character is (: Push it onto the stack.
3. If the character is): Pop operators from the stack and append them to the output until a matching (is encountered. Pop and discard the (.
4. If the character is an operator:
   - While the stack is not empty and the operator at the top of the stack has greater than or equal precedence to the current operator, pop it and append it to the output.
   - Push the current operator onto the stack.
5. End of Expression: Pop all remaining operators from the stack and append them to the output.

Code Implementation:

```
#include <iostream>

#include <stack>

#include <string>

using namespace std;

int precedence(char op) {
    if (op == '^') return 3;
    if (op == '*' || op == '/') return 2;
    if (op == '+' || op == '-') return 1;
```

```
    return 0;
}

bool isOperand(char c) {
    return isalnum(c);
}

string infixToPostfix(string expr) {
    stack < char > st;
    string result = "";
    for (char c: expr) {
        if (isOperand(c)) {
            result += c;
        } else if (c == '(') {
            st.push(c);
        } else if (c == ')') {
            while (!st.empty() && st.top() != '(') {
                result += st.top();
                st.pop();
            }
            if (!st.empty()) st.pop(); // remove '('
        } else {
            while (!st.empty() && precedence(st.top()) >= precedence(c)) {
                result += st.top();
                st.pop();
            }
            st.push(c);
        }
    }
    while (!st.empty()) {
        result += st.top();
        st.pop();
    }
    return result;
}

int main() {
    string expr;
    cout << "Enter infix expression: ";
    cin >> expr;
    cout << "Postfix: " << infixToPostfix(expr) << endl;
    return 0;
}
```

Sample Input

```
A+B*(C/D)-E
```

Your Output

```
Enter infix expression: Postfix: ABCD/*+E-
```

Practical 3

Aim: Implement Postfix evaluation using Stack.

Theory:

In computer science, arithmetic expressions can be written in three main notations: **Infix** (e.g., A + B), **Prefix** (e.g., + A B), and **Postfix** (e.g., A B +).

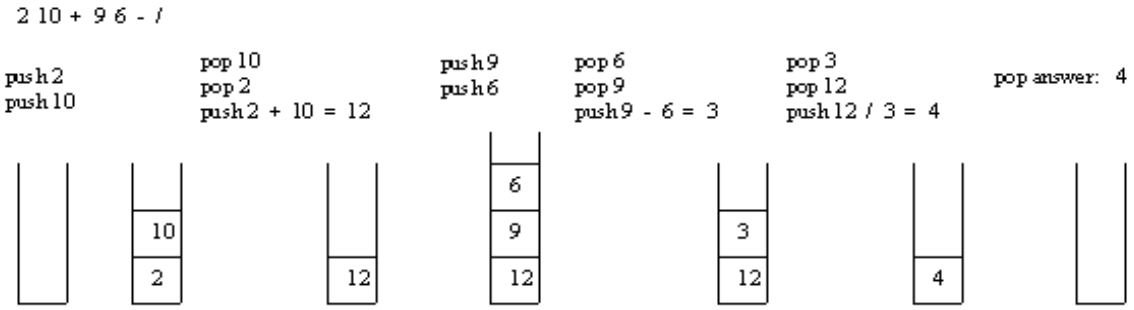
While humans prefer Infix notation, it is computationally expensive for machines because it requires scanning the expression multiple times to resolve operator precedence (BODMAS/PEMDAS) and parentheses. **Postfix notation** (Reverse Polish Notation) eliminates the need for parentheses entirely. The operator precedence is strictly determined by the order of elements. Because operators appear immediately after their operands, a compiler can evaluate a postfix expression in a single left-to-right pass using a **Stack**.

Algorithmic Steps:

1. Initialize an empty stack capable of holding numerical values.
2. Scan the postfix string from left to right.
3. If the scanned character is an **operand** (a number or variable), push it onto the stack.
4. If the scanned character is an **operator** (+, -, *, /, ^):
    - Pop the top element from the stack and store it as operand2.
    - Pop the next top element and store it as operand1. *(Note: The order of popping is critical for non-commutative operations like subtraction and division).*
    - Perform the operation: operand1 operator operand2.
    - Push the computed result back onto the stack.
5. Once the expression ends, the single value remaining in the stack is the final answer.

Example Trace for 5 3 + 8 2 - * :

- Push 5, Push 3. (Stack: 5, 3)
- Read +: Pop 3, Pop 5. Compute 5 + 3 = 8. Push 8. (Stack: 8)
- Push 8, Push 2. (Stack: 8, 8, 2)
- Read -: Pop 2, Pop 8. Compute 8 - 2 = 6. Push 6. (Stack: 8, 6)
- Read *: Pop 6, Pop 8. Compute 8 * 6 = 48. Push 48. (Stack: 48)
- Result = 48.

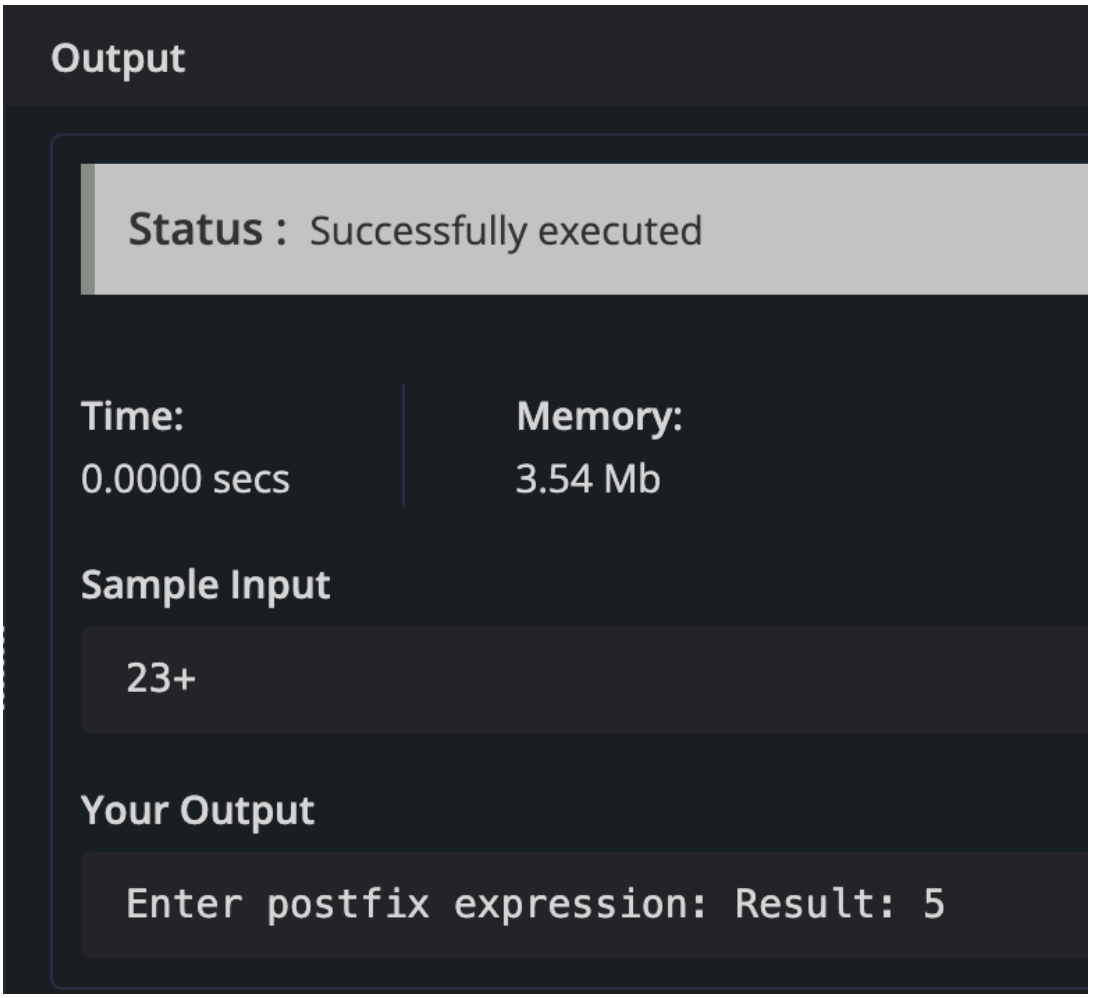


Code Implementation:

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

int evaluatePostfix(string expr) {
   stack<int> st;
   for (char c : expr) {
      if (isdigit(c)) {
         st.push(c - '0');
      } else {
         int b = st.top(); st.pop();
         int a = st.top(); st.pop();
         switch (c) {
            case '+': st.push(a + b); break;
            case '-': st.push(a - b); break;
            case '*': st.push(a * b); break;
            case '/': st.push(a / b); break;
            case '^': {
               int res = 1;
               for (int i = 0; i < b; i++) res *= a;
               st.push(res); break;
            }
         }
      }
   }
   return st.top();
}
```

```
int main() {
    string expr;
    cout << "Enter postfix expression: ";
    cin >> expr;
    cout << "Result: " << evaluatePostfix(expr) << endl;
    return 0;
}
```



Practical 4

Aim: Implement Towers of Hanoi using Stack.

Theory:

The **Towers of Hanoi** is a mathematical puzzle involving three pegs—Source (S), Auxiliary (A), and Destination (D)—and n disks of distinct sizes. The goal is to move all disks from the Source peg to the Destination peg. The fundamental constraints are:

1. Only the topmost disk on any peg can be moved.
2. You cannot place a larger disk on top of a smaller disk.

The optimal number of moves to solve the puzzle for n disks is always **$2^n - 1$**.

The Iterative Approach Using a Stack:

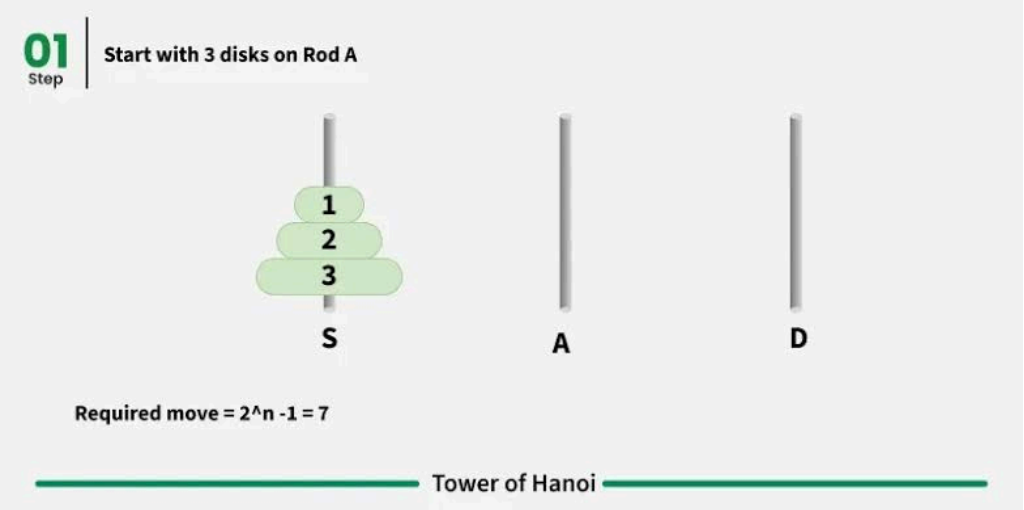
While typically solved using recursion, the iterative method relies on calculating the total number of moves in advance and establishing a cyclic movement pattern. We simulate the recursive function calls by maintaining the state of the pegs using custom stack data structures.

The algorithm behaves differently based on whether the number of disks (n) is even or odd:

- **If n is even:** The sequence of valid moves cycles between: S ↔ A, S ↔ D, and A ↔ D.
- **If n is odd:** The sequence of valid moves cycles between: S ↔ D, S ↔ A, and A ↔ D.

Algorithm:

1. Calculate the total number of moves: total_moves = pow(2, n) - 1.
2. Run a loop from i = 1 to total_moves.
3. In each iteration, based on i % 3, perform the only valid move between the two designated pegs.
4. A "valid move" between Peg X and Peg Y means:
    - If Peg X is empty, pop from Y and push to X.
    - If Peg Y is empty, pop from X and push to Y.
    - If neither is empty, compare their top disks. Pop the smaller disk and push it onto the peg with the larger disk.



Code Implementation:

```
#include <iostream>
using namespace std;

void hanoi(int n, char src, char dest, char aux) {
    if (n == 1) {
        cout << "Move disk 1 from " << src << " to " << dest << "\n";
        return;
    }
    hanoi(n - 1, src, aux, dest);
    cout << "Move disk " << n << " from " << src << " to " << dest << "\n";
    hanoi(n - 1, aux, dest, src);
}

int main() {
    int n;
    cout << "Enter number of disks: ";
    cin >> n;
    cout << "\nSteps to solve Tower of Hanoi with " << n << " disks:\n";
    hanoi(n, 'A', 'C', 'B');
    cout << "\nTotal moves: " << ((1 << n) - 1) << "\n";
    return 0;
}
```

Sample Input

```
2
```

Your Output

```
Enter number of disks:
Steps to solve Tower of Hanoi with 2 disks:
Move disk 1 from A to B
Move disk 2 from A to C
Move disk 1 from B to C

Total moves: 3
```

Practical 5

Aim: Implement queue and its operations like enqueue, dequeue, traverse, search.

Theory:

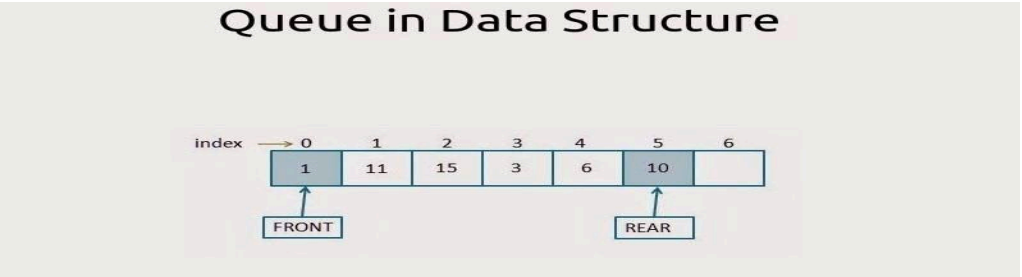
A **Queue** is a linear data structure that models real-world waiting lines (like a printer queue or CPU task scheduling). It strictly enforces a **First-In-First-Out (FIFO)** ordering. Elements are inserted at one end and removed from the other.

To manage this, a queue uses two pointer variables:

- **Front:** Tracks the index of the first element (the one to be removed next).
- **Rear:** Tracks the index of the last inserted element. Initially, both `Front` and `Rear` are set to `-1` to denote an empty queue.

Detailed Operations:

1. **Enqueue (Insertion):**
   - **Check Overflow:** Check if `Rear == MAX_SIZE - 1`. If true, the queue is full.
   - If inserting the very first element, set both `Front = 0` and `Rear = 0`.
   - Otherwise, increment `Rear` by 1 and store the new value at `Queue[Rear]`.
2. **Dequeue (Deletion):**
   - **Check Underflow:** Check if `Front == -1` or `Front > Rear`. If true, the queue is empty.
   - Retrieve the value at `Queue[Front]`.
   - Increment `Front` by 1. *(Note: in a basic linear queue, this "wastes" the space at the beginning of the array, which is why Circular Queues were invented).*
3. **Traverse:** Loop from the `Front` index to the `Rear` index, printing each element sequentially.
4. **Search:** Iterate from `Front` to `Rear` comparing each element to a target value. Return the position if found.



Code Implementation:

```
#include <iostream>
using namespace std;

#define MAX 100

class Queue {
   int arr[MAX];
   int front, rear;
public:
   Queue() { front = -1; rear = -1; }

   bool isEmpty() { return front == -1; }
   bool isFull()  { return rear == MAX - 1; }

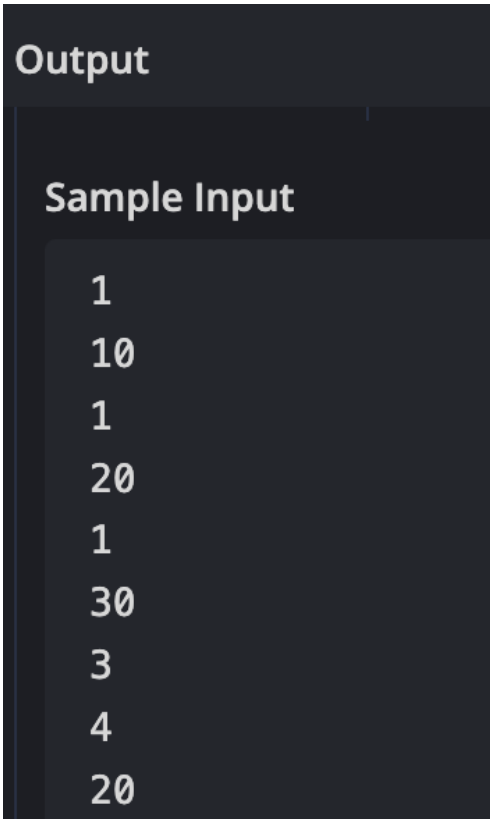
   void enqueue(int val) {
      if (isFull()) { cout << "Queue is Full!\n"; return; }
      if (isEmpty()) front = 0;
      arr[++rear] = val;
      cout << val << " enqueued.\n";
   }

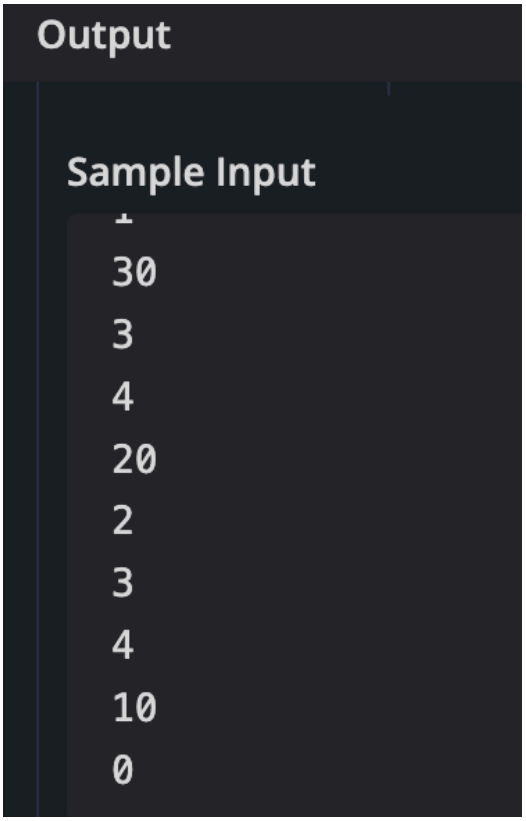
   void dequeue() {
      if (isEmpty()) { cout << "Queue is Empty!\n"; return; }
      cout << arr[front] << " dequeued.\n";
      if (front == rear) { front = rear = -1; }
      else front++;
   }

   void traverse() {
      if (isEmpty()) { cout << "Queue is empty.\n"; return; }
      cout << "Queue (front to rear): ";
      for (int i = front; i <= rear; i++) cout << arr[i] << " ";
      cout << "\n";
   }

   void search(int val) {
      for (int i = front; i <= rear; i++)
         if (arr[i] == val) {
            cout << val << " found at position " << (i - front + 1) << ".\n";
            return;
         }
      cout << val << " not found.\n";
   }
};
```

```
int main() {
    Queue q;
    int choice, val;
    do {
        cout << "\n--- Queue Menu ---\n";
        cout << "1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit\n";
        cout << "Enter choice: "; cin >> choice;
        switch (choice) {
            case 1: cout << "Value: "; cin >> val; q.enqueue(val); break;
            case 2: q.dequeue(); break;
            case 3: q.traverse(); break;
            case 4: cout << "Search: "; cin >> val; q.search(val); break;
        }
    } while (choice != 0);
    return 0;
}
```





Your Output

```
--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Value: 10 enqueued.

--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Value: 20 enqueued.
```

Your Output

```
--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Value: 30 enqueued.

--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Queue (front to rear): 10 20 30

--- Queue Menu ---
```

Your Output

```
--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Search: 20 found at position 2.

--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: 10 dequeued.

--- Queue Menu ---
```

Your Output

```
Enter choice: 10 dequeued.

--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Queue (front to rear): 20 30

--- Queue Menu ---
1.Enqueue 2.Dequeue 3.Traverse 4.Search 0.Exit
Enter choice: Search: 10 not found.
```

Practical 6

Aim: Implement Single Linked lists and its operations (creation insertion deletion traversal search reverse)

Theory:

Unlike arrays, which require contiguous memory allocation, a **Singly Linked List** allocates memory dynamically at runtime. It consists of independent blocks of memory called **Nodes**. Each node contains two fields:

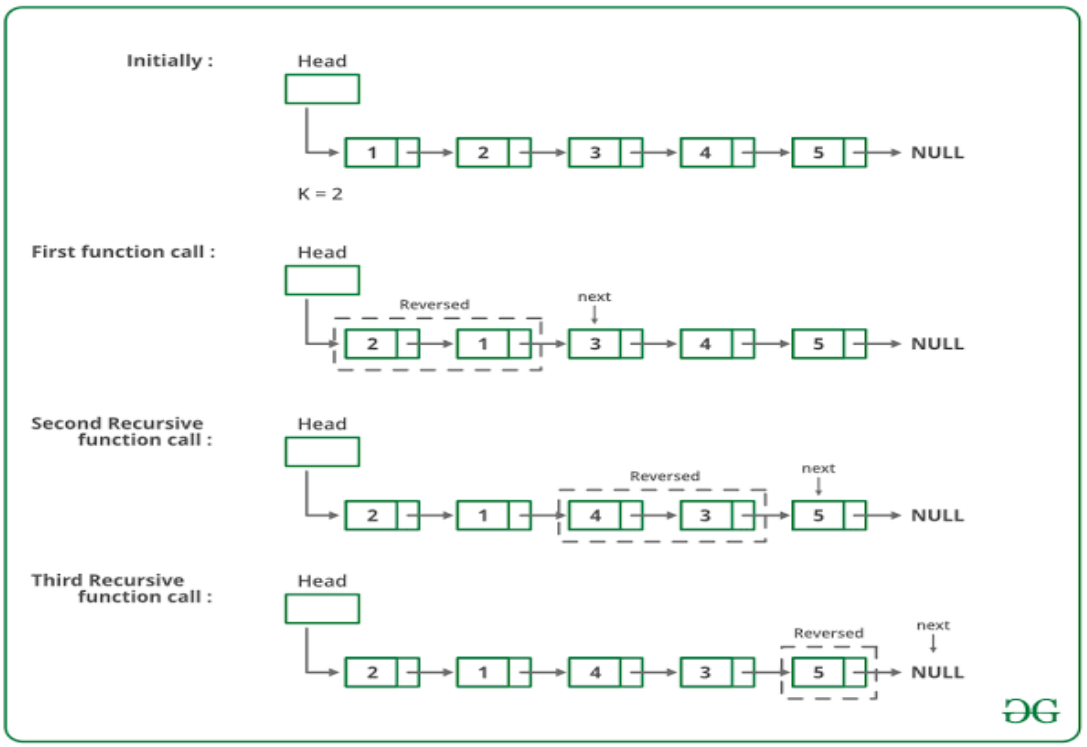
1. **Data:** The actual information.
2. **Next:** A pointer holding the memory address of the next node.

The list is accessed via a `Head` pointer, which points to the first node. The `Next` pointer of the final node is set to `NULL`, marking the end of the list.

Detailed Operations:

- **Creation & Memory Allocation:** To create a new node in C/C++, use `malloc()` or `new`. Assign the data and set its `next` to `NULL`.
- **Insertion:**
    1. *At Beginning:* Set the new node's `next` to point to the current `Head`. Then, update `Head` to point to the new node.
    2. *At End:* Traverse the list until `current->next == NULL`. Set `current->next` to point to the new node.
- **Deletion:**
    1. To delete a node, you must track the `previous` node. Set `previous->next = node_to_delete->next`. Then, explicitly free the memory of `node_to_delete` to prevent memory leaks.
- **Traversal & Search:** Initialize a temporary pointer `temp = Head`. Use a `while(temp != NULL)` loop to visit each node, process its data, and then step forward using `temp = temp->next`.
- **Reverse:** Reversing a linked list requires a three-pointer sliding window logic (`prev`, `current`, `next`).
    1. Initialize `prev = NULL` and `current = Head`.
    2. In a loop, temporarily store `next = current->next`.
    3. Flip the current node's pointer backwards: `current->next = prev`.

4. Slide the window forward: `prev = current` and `current = next`.
5. When the loop finishes, update `Head = prev`.



Code Implementation:

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};

class SLL {
    Node* head;
public:
    SLL() : head(nullptr) {}

    void insertAtBegin(int val) {
        Node* n = new Node(val);
        n->next = head; head = n;
        cout << val << " inserted at beginning.\n";
    }
```

```
    void insertAtEnd(int val) {
        Node* n = new Node(val);
        if (!head) { head = n; }
        else {
            Node* temp = head;
            while (temp->next) temp = temp->next;
            temp->next = n;
        }
        cout << val << " inserted at end.\n";
    }

    void deleteNode(int val) {
        if (!head) { cout << "List empty.\n"; return; }
        if (head->data == val) {
            Node* temp = head; head = head->next; delete temp;
            cout << val << " deleted.\n"; return;
        }
        Node* prev = head, *curr = head->next;
        while (curr && curr->data != val) { prev = curr; curr = curr->next; }
        if (!curr) { cout << val << " not found.\n"; return; }
        prev->next = curr->next; delete curr;
        cout << val << " deleted.\n";
    }

    void traverse() {
        Node* temp = head;
        cout << "List: ";
        while (temp) { cout << temp->data << " -> "; temp = temp->next; }
        cout << "NULL\n";
    }

    void search(int val) {
        Node* temp = head; int pos = 1;
        while (temp) {
            if (temp->data == val) { cout << val << " found at position " << pos << "\n"; return; }
            temp = temp->next; pos++;
        }
        cout << val << " not found.\n";
    }

    void reverse() {
        Node *prev = nullptr, *curr = head, *next = nullptr;
        while (curr) { next = curr->next; curr->next = prev; prev = curr; curr = next; }
        head = prev;
        cout << "List reversed.\n";
    }
};

int main() {
    SLL list;
```

```
    int ch, val;
    do {
        cout << "\n1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit\n";
        cout << "Choice: "; cin >> ch;
        switch(ch) {
            case 1: cout<<"Val: "; cin>>val; list.insertAtBegin(val); break;
            case 2: cout<<"Val: "; cin>>val; list.insertAtEnd(val); break;
            case 3: cout<<"Val: "; cin>>val; list.deleteNode(val); break;
            case 4: list.traverse(); break;
            case 5: cout<<"Search: "; cin>>val; list.search(val); break;
            case 6: list.reverse(); break;
        }
    } while(ch != 0);
    return 0;
}
```

Sample Input

```
1
10
2
20
2
30
1
5
4
```

Sample Input

```
4
5
20
6
4
3
20
4
0
```

Your Output

```
1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 10 inserted at beginning.

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 20 inserted at end.

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 30 inserted at end.
```

Your Output

```
1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 5 inserted at beginning.

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: List: 5 -> 10 -> 20 -> 30 -> NULL

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Search: 20 found at position 3
```

Your Output

```
1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: List reversed.

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: List: 30 -> 20 -> 10 -> 5 -> NULL

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 20 deleted.
```

Your Output

```
Choice: List: 30 -> 20 -> 10 -> 5 -> NULL

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: Val: 20 deleted.

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice: List: 30 -> 10 -> 5 -> NULL

1.InsertBegin 2.InsertEnd 3.Delete 4.Traverse 5.Search 6.Reverse 0.Exit
Choice:
```